시스템 분석 및 설계

04주차. 기능 요구 분석

서울사이버대학교

학습목표

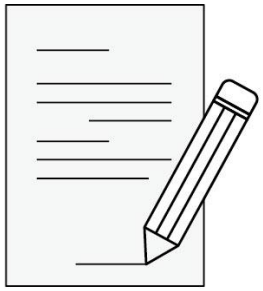
1. 기능 요구 분석법에서 사용되는 기법을 이해할 수 있다.
2. 자료 흐름도(DFD)의 개념과 작성법을 이해할 수 있다.
3. 자료 사전과 프로세스 명세서에 포함되는 요소를 식별할 수 있다.
4. 유즈 케이스 다이어그램의 개념과 사용법을 이해할 수 있다.
5. 요구 분석 명세서에 포함되어야 하는 내용과 작성시 주의점을 이해할 수 있다.

기능요구 분석법(Functional Requirements Analysis)은 시스템 또는 소프트웨어 개발 과정에서 사용자의 요구사항을 명확히 정의하고, 이를 바탕으로 시스템이 수행해야 할 기능을 분석하는 방법

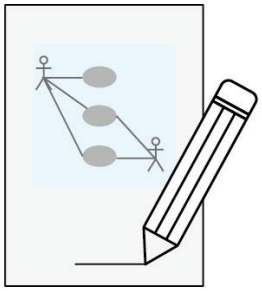
- 시스템의 목표를 달성하기 위해 필요한 기능을 식별하고, 이를 체계적으로 정리하여 개발자, 설계자, 이해관계자 간의 명확한 의사소통을 돕는 데 목적이 있음
- 고객의 의도를 묻고 요구를 정리하여 전달하는 등 의사 교환 작업이 대부분

다이어그램을 이용한 방법은 일반적으로 기능 요구를 표현하는데 사용

다이어그램은 요구를 표현하는 그림 요소들로 이뤄지며, 나중에 소프트웨어의 기능 요소들로 매핑됨

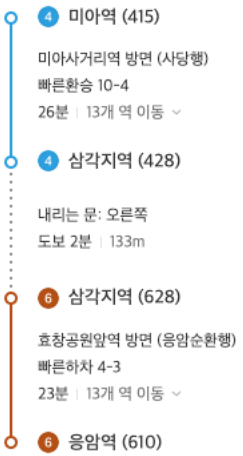


언어를 이용한 방법
• 장점 : 배울 필요가 없음(고객이 쉽게 이해), 표현에 제한이 없음. • 단점 : 정확도가 떨어짐, 모호할 수 있고 도구가 없음.



다이어그램을 이용한 방법
• 장점 : 복잡한 문제나 해결책 이해에 도움이 됨. 커뮤니케이션이 용이함. 구현에 이용할 수 있음. • 단점 : 제한된 표현, 배워야 함.

유닛1	기능 요구 분석	페이지2	다이어그램의 장점



지도로 볼 때 (시각화 중심)

장점

- 직관성: 사용자는 지도를 통해 전체적인 구조를 빠르게 이해
- 공간적 관계 파악: 역 간의 거리, 위치, 환승 지점 등을 쉽게 파악
- 시각적 기억: 색상과 형태로 정보가 제공되기 때문에 기억 용이

단점

- 복잡성: 노선이 많을 경우 지도가 복잡해져 정보를 파악하기 어려움
- 세부 정보 부족: 지도는 전체적인 구조를 보여주지만, 특정 역의 출구 정보나 세부 시간표 등은 제공하지 않음

텍스트 안내로 볼 때 (상세화 중심)

장점

- 정확성: 텍스트는 명확하고 구체적인 정보를 제
- 세부 정보 제공: 지도에서는 표현하기 어려운 세부 정보(예: 출구 번호, 주변 시설)를 제공
- 접근성: 시각적 제약이 있는 사용자도 텍스트를 통해 정보를 얻을 수 있음

단점

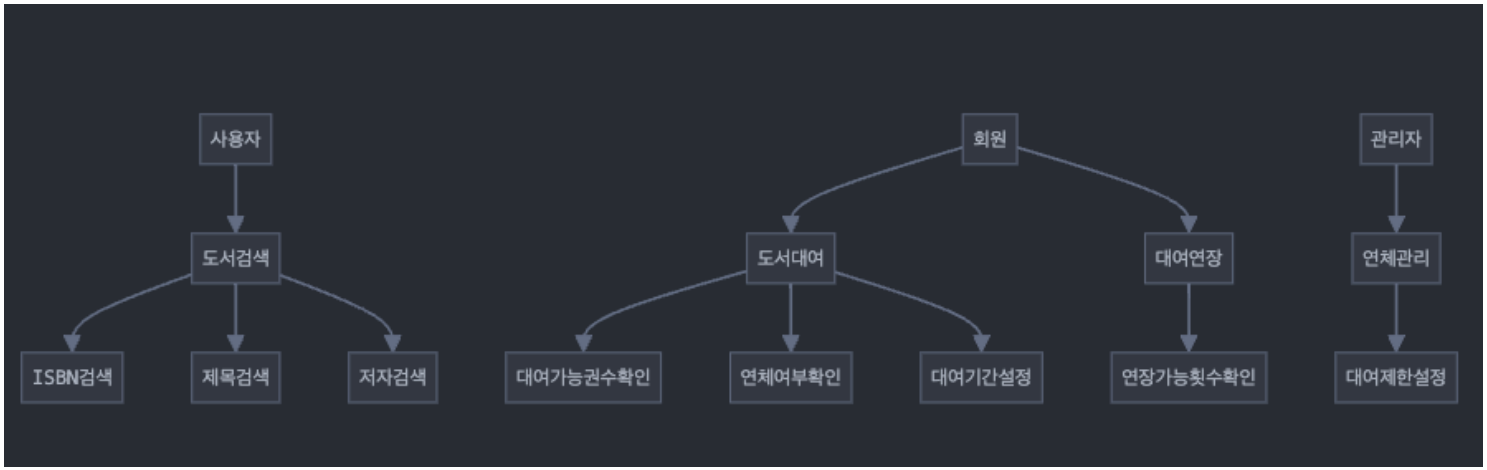
- 비직관성: 텍스트는 전체적인 구조를 한눈에 파악하기 어려움
- 시간 소요: 정보를 순차적으로 읽어야 하기 때문에 빠르게 이해하기 어려움
- 기억 어려움: 텍스트는 시각적 정보보다 기억하기 어려움

도서 대여 시스템의 예

언어(텍스트) 기반 설명

- 사용자는 도서 검색이 가능해야 한다
  - ISBN, 제목, 저자명으로 검색할 수 있다 –
  - 검색 결과는 도서명, 저자, 출판사, 대여가능 여부를 포함한다
- 회원은 도서 대여가 가능해야 한다
  - 한 번에 최대 3권까지 대여할 수 있다
  - 대여 기간은 14일이며, 1회 연장 가능하다
  - 연체 시 연체일수만큼 대여가 제한된다

다이어 그램 기반 설명



소프트웨어 기능 전체를 파악하고 이해하기 위한 다이어그램은 어떤 요소를 기반으로 하는지에 따라 다양한 방법

- 흐름 기반 방법: DFD, 순서도
- 시나리오 기반 방법: 유스 케이스 다이어그램, 액티비티 다이어그램
- 동작 기반 방법: 상태 다이어그램, 시퀀스 다이어그램

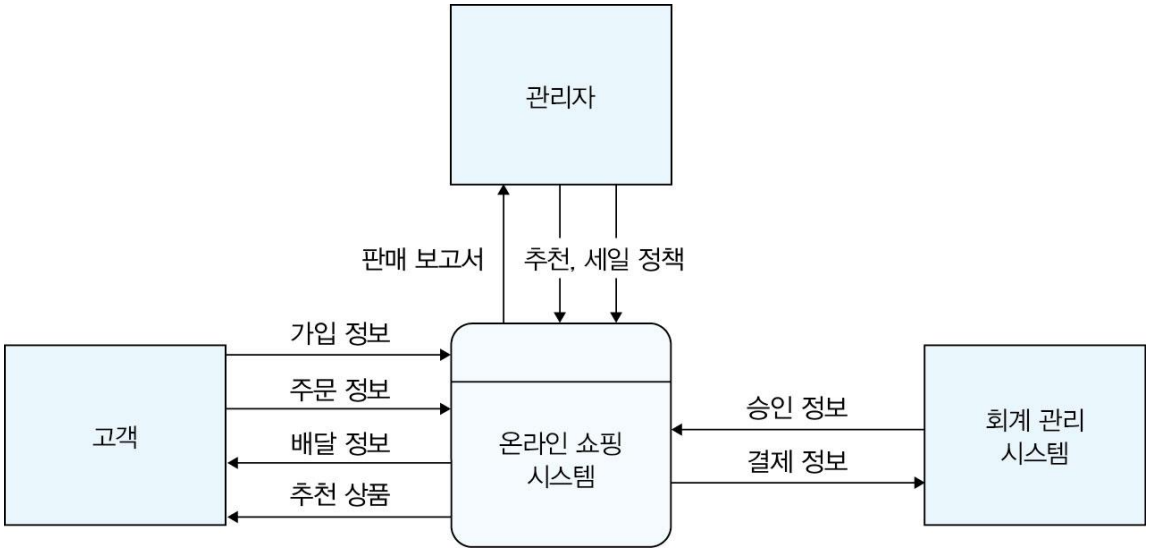
요구 분석 단계에 사용되는 다이어그램은 요구 기능 자체 뿐 아니라 소프트웨어 기능의 큰 덩어리를 파악하는데 사용되기 때문에 업무 흐름도와는 다름, 즉 소프트웨어 구성 요소의 관점으로 기능 요구를 표현한 것

- DFD (Data Flow Diagram, 데이터 흐름도)는 시스템 내에서 데이터가 어떻게 이동하고, 변환되고, 저장되는지를 도식화한 그래픽 모델링 기법입니다. 시스템을 구성하는 프로세스들 간의 데이터 흐름과 데이터 저장소를 중심으로 시스템의 동적인 측면을 표현하는 기법
  - 구조적 분석 방법에서 기능 요구를 찾아내기 위해 널리 사용되는 방법
  - 구조적 분석에서는 시스템을 자료가 변환해 나가는 과정으로 보고 DFD를 그리면서 변환과정에서 어떤 기능이 필요한지 찾아냄
  - 시스템에 어떤 자료가 입력되어 출력정보로 가공되는지에 초점을 두는 다이어그램, 즉 자료를 가공하는 기능 위주로 시스템의 요소를 분할하는 방법
- 다음과 같은 특징을 지님
  - 센서 같은 하드웨어나 구현 방법에 관계없이 자료의 처리 흐름만을 나타내는 논리적 기법
  - 상세한 계산 방법이나 제어 흐름이 아니라 어떤 자료가 처리되는지에 초점
  - 자료가 변환되어가는 흐름을 나타내지만 순서도와 같이 조건에 따른 반복, 분기 흐름은 표현하지 않음

- 개발하는 소프트웨어의 DFD를 그리는 과정은 단계적으로 분할하는 원리를 잘 따르게 됨
- 처음에 시스템 전체를 하나의 기능 덩어리로 보고 DFD를 그린 후 레벨1에서는 시스템에 존재하는 큰 기능으로 나누어 보고, 각 기능에 어떤 자료가 들어오고, 어디에 보관되며, 어떤 자료로 가공되어 나가는지 나타냄

[스텝1]

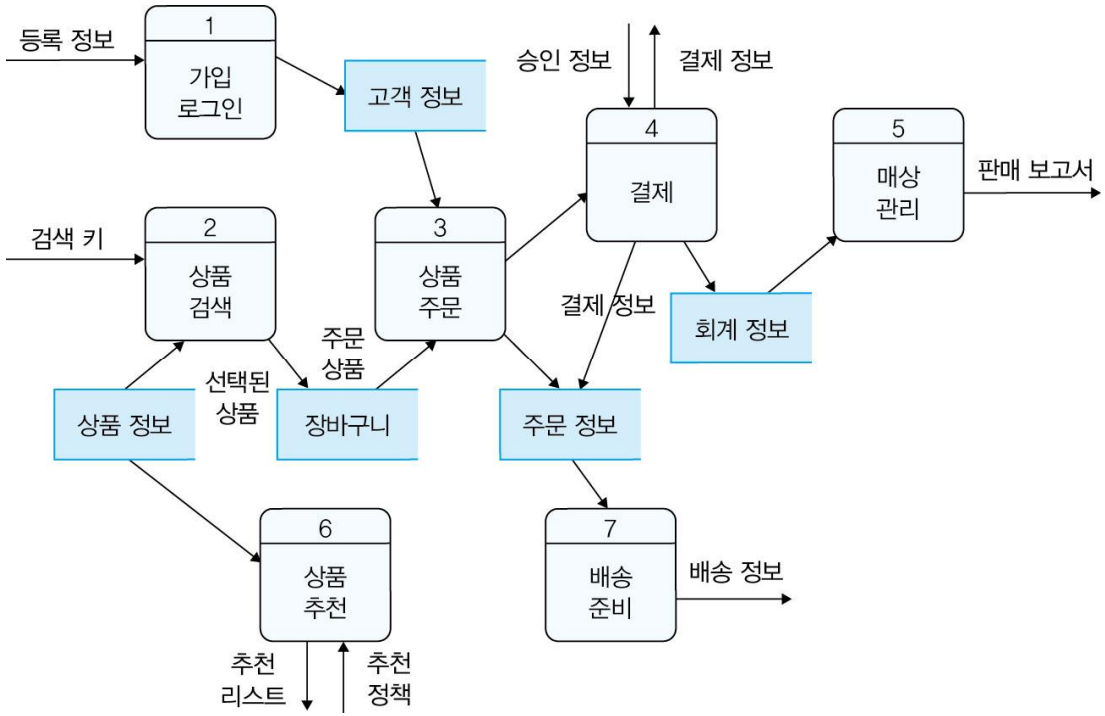
- 배경도는 소프트웨어의 범위를 나타냄
- 자료를 입력받고, 제공받는 외부 출처를 나타내며
- 어떤 자료와 정보가 들어오고 나가는지 표시



배경도

[스텝2]

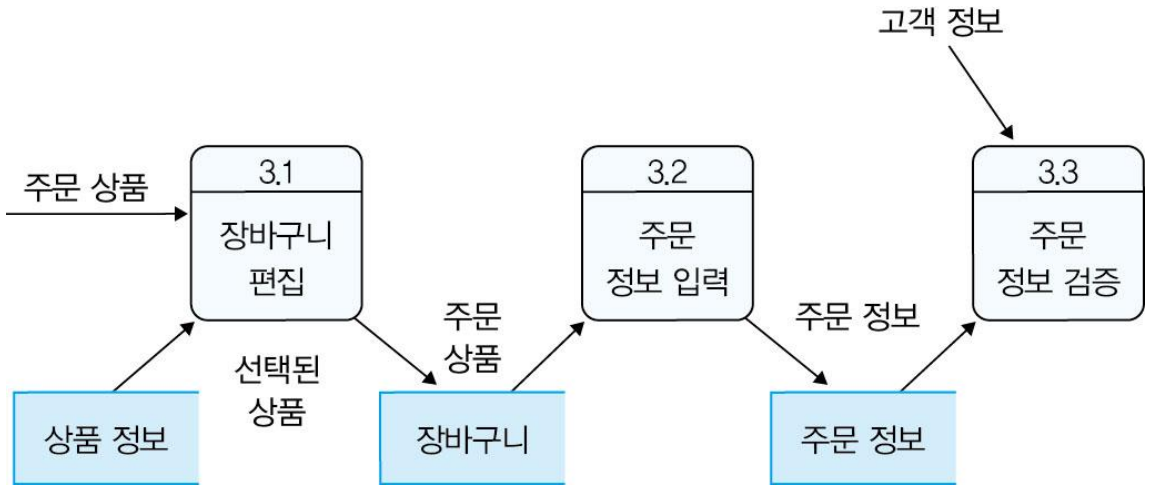
- 레벨1 DFD는 배경도의 프로세스 안에 있는 하위 프로세스를 함께 표시
- 여기 포함된 프로세스들이 모여 완전한 시스템을 형성한다는 의미
- 결국 레벨1 DFD는 배경도가 더 자세히 분해된 그림
- 레벨1에는 배경도에 나타내는 외부 요소를 꼭 나타낼 필요는 없지만, 배경도의 외부에서 들어오고 나오는 자료의 흐름은 레벨1에서 모두 표시되어야 함



레벨1 DFD

[스텝3]

- 이미 레벨0 배경도를 레벨1 DFD로 세분화하는 방법을 배웠음
- 이 때 배경도를 부모로, 레벨1 다이어그램을 하위DFD로 볼 수 있음
- 동일한 과정을 레벨1 DFD에 적용하여, 레벨2를 그려 나감, 즉 레벨1 내에 있는 각 프로세스에 적용될 수 있음
- 분할된 레벨2의 DFD프로세스에는 그 소속을 알 수 있도록 번호를 부여함.
- 예를 들어 레벨1의 <상품 주문> 프로세스를 분할한 레벨2 DFD에는 그림과 같이 3.x 형식의 번호를 부여



레벨2 DFD

이렇게 각 프로세스에 대한 분할을 반복하다가 프로세스가 더 분할하기 어려운 최소 업무단위(작업)이 되면 멈춤

분할이 멈춘 프로세스에 대해 내부 알고리즘을 미니스펙으로 작성하는 것으로 완성

요약하면,

- 처음에 배경도를 그리는 것을 시작으로 레벨1 다이어그램을 한 장 그림 → 각 프로세스를 분할하여 다음 레벨의 DFD를 그림
- 다음 레벨의 다이어그램을 그릴 때 주의할 점은 분할하는 프로세스에 들어오고 나가는 자료의 흐름이 빠짐없이 표시되어야 함, 이를 자료 흐름의 균형이라고 함

1. 프로세스 식별: 부모 프로세스 내부에 있어야 할 작은 기능들을 프로세스로 나타냄
2. 자료흐름 파악
   - 각 프로세스에 입력, 출력되는 자료 흐름을 파악하여 나타냄
   - 각 프로세스에는 하나 이상의 출력 자료 흐름이 있어야 함
   - 외부 엔티티와 프로세스 간의 자료 흐름은 하위 DFD에서도 일치하도록 그림
3. 자료 저장소 식별
   - 문서, 파일 등 자료 저장소를 추가하고 이름을 붙인 후 프로세스와 자료 저장소 사이에 이동하는 자료 흐름을 추가
   - 각 자료 저장소에는 적어도 하나의 입력 데이터 흐름과 하나의 출력 데이터 흐림이 있어야 함
4. 다이어그램 검토
   - 각 프로세스에는 입력과 출력이 있어야 함
   - 각 데이터 저장소에는 입력과 출력이 있어야 함
   - 구동시키는 자료 흐름 없이 진행 중인 프로세스가 있는지 확인함

DFD는 개별 기능을 프로세스로 표현함

- 프로세스를 더 이상 나눌 수 없는 경우 프로세스 내부의 처리 내용을 자세히 설명할 수 없음
- 따라서 상세한 처리 내용을 프로세스 명세로 기술

프로세스 안에서 실행할 내용을 수집하고 실행하는 순서를 규정하면 이것이 소프트웨어의 동작이 됨

- 프로세스 명세는 구조적 언어를 이용하여 기술함
- 구조적 언어는 단순문, 선택구문, 반복구문 등의 조합으로 이뤄짐

프로세스 명세: 투입된 금액 계산
입력: 투입된 동전 정보
출력: 투입 금액
처리: for 투입된 동전의 정보가 없어질 때까지 다음을 반복
  if (동전의 정보 == 10원) 10원짜리 개수 = 10원짜리 개수 + 1
  if (동전의 정보 == 50원) 50원짜리 개수 = 50원짜리 개수 + 1
  if (동전의 정보 == 100원) 100원짜리 개수 = 100원짜리 개수 + 1
  if (동전의 정보 == 500원) 500원짜리 개수 = 500원짜리 개수 + 1
  투입 금액 = 10원짜리 개수×10 + 50원짜리 개수×50 +
    100원짜리 개수×100 + 500원짜리 개수×500

구조적 언어로 기술한 자동판매기의 투입 금액을 계산하는 프로세스 명세의 예

투입된 동전의 종류에 따라 해당 금액을 총액에 가산

자료 사전은 DFD 및 프로세스 명세에서 사용되는 자료 흐름, 자료 저장소의 이름, 자료 유형, 단위, 범위, 정확도를 일괄적으로 관리하기 위한 데이터베이스

자료 사전을 통해 DFD 및 프로세스 명세를 보다 엄밀하게 관리할 수 있음

자료 사전에 사용되는 기호

심볼 기호	설명
a = ~	자료의 정의, 자료 a는 ~와 같이 정의됨.
A = ~	집합 자료의 정의, 자료 집합 A는 ~와 같이 정의됨.
A + B	자료의 연결, A 자료 후속으로 B가 나옴.
$\{A\}_n^m$	자료의 반복, A가 최소 n개에서 최대 m개까지 반복됨.
[A | B]	자료의 선택, A와 B 중 하나가 선택되어 출현함.
(A)	자료의 생략, A가 존재할 수도 있고 없을 수도 있음.
'A'	문자 A
A	A는 주석(설명)

- UML에 포함된 유스 케이스 다이어그램의 목적은 시스템이 어떤 서비스를 제공하는지 찾아내어 캡처하기 위한 것
- 또한 시스템 내부와 외부 영향을 포함하여 시스템의 요구 사항을 수집하는데 사용
- 기능을 수집하기 위해 유스 케이스를 찾아내고 액터를 식별하게 됨
- 유스 케이스는 사용자가 본 소프트웨어 사용의 전형적인 예이며 소프트웨어에 대한 요구, 즉 제공하는 기능을 파악하기 위해 사용
- 소프트웨어에 존재하는 유스 케이스와 이를 이용하는 사용자와의 관계를 나타내는 그림, 즉 소프트웨어의 기능과 사용자, 또는 다른 시스템과 같은 외부 요소와의 관계를 표현하는 기법

스텝 1 - 시스템의 외부 요소(액터)를 파악

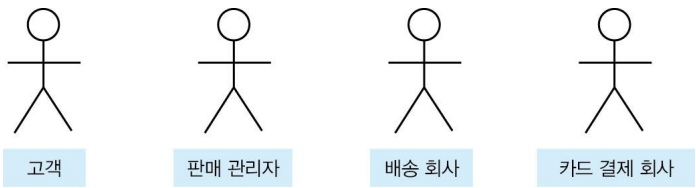
- 유스케이스 다이어 그램은 액터와 소프트웨어 시스템의 경계, 즉 개발 대상인 소프트웨어의 범위를 나타냄
- 경계를 나타내는 박스는 생략할 수 있지만, 액터는 반드시 표시해야 하는 대상으로, 소프트웨어와 상호작용하는 사용자나 다른 시스템을 의미
- 소프트웨어의 경계 안에는 대상 시스템에서 사용 가능한 유스 케이스의 집합을 정의
- 액터는 대상과 상호작용하는 실체이기 때문에 사용자 뿐 아니라 대상 시스템과 연계하는 다른 시스템도 포함하며, 막대 인간 모양으로 표시

스텝 2 – 시스템의 내부 요소인 유스 케이스를 파악

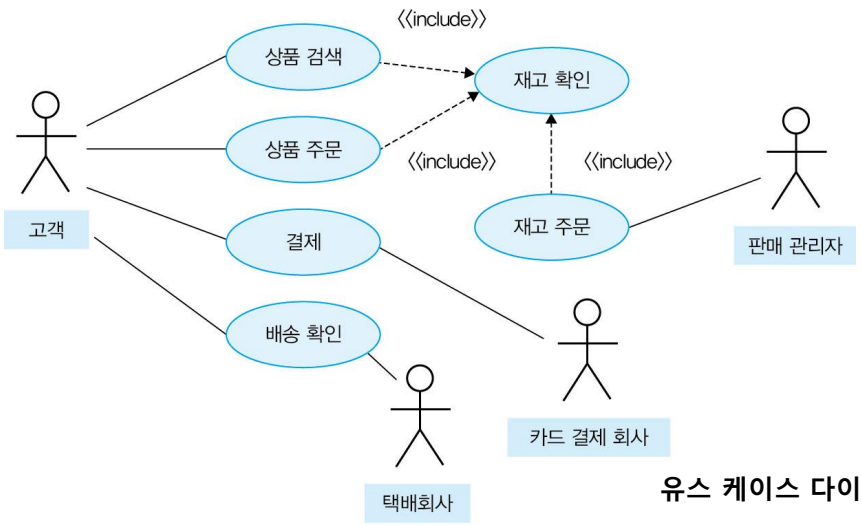
- 유스 케이스는 외부에 보이는 시스템의 기능
- 따라서 상세하거나 규모가 작은 것은 피하고 매우 높은 수준의 추상적인 기능 요구를 파악
- 유스 케이스 다이어그램에서 유스케이스는 타원으로 나타내고, 그 아래에 유스 케이스의 이름을 붙임
- 유스 케이스와 연관된 액터 사이의 관계는 실선으로 표시

스텝 3 – 유스 케이스 사이의 관계를 파악하여 표현

- 여러 유스 케이스 사이에는 중복된 절차가 있을 수 있음 → 공통부분을 하나의 유스 케이스로 정의
- 이렇게 정의하면 하나의 유스 케이스가 중복된 유스 케이스를 포함할 수 있음 → 이 때 다이어그램에서는 둘 사이의 관계를 <<include>>로 표시
- 기존 유스 케이스에 몇 가지 단계를 추가하여 새로운 유스 케이스를 만들 수 있음 → 유스 케이스의 확장
- 확장의 대상이 되는 유스 케이스에는 확장점이 지정되며, 점선 화살표 위에 <<extend>> 표기



인터넷 쇼핑몰 시스템의 액터



유스 케이스 다이어그램

DFD	유스 케이스
• 어떤 변환이 이뤄지는지에 초점을 두며, 어떻게 이뤄지는지는 중요하지 않음 • 중요한 입력과 출력이 표현되며 사소한 내용은 생략됨 • 반복, 조건 흐름이 없음 • 제어 흐름도나 알고리즘식 설계가 아님	• 사용자와 시스템 사이에 이뤄지는 동작에 대하여 캡처한 것 • 기본적으로는 텍스트 형식이며, 다이어그램은 보조적 • 요구의 추출과 문제 분석에 사용 • 브레인스토밍이나 기능 이해를 위한 소통 도구로도 적절

온라인 쇼핑몰 시스템의 경우

DFD 사용이 적절한 상황

- 주문 데이터 생성 후 결제, 배송, 재고 관리 시스템으로 흘러가는 과정 분석 할 때
- 상품 정보가 데이터베이스에 저장되고 검색되는 프로세스를 설계할 때
- 회원 정보의 수집, 처리, 저장 흐름을 분석할 때

유스케이스 다이어그램 사용이 적절한 상황

- 고객이 상품을 검색하고 구매하는 시나리오를 정의할 때
- 판매자가 상품을 등록하고 관리하는 기능을 설계할 때
- 관리자가 주문을 처리하고 배송을 관리하는 업무를 정의할 때

병원 정보 시스템의 경우

DFD 사용이 적절한 상황

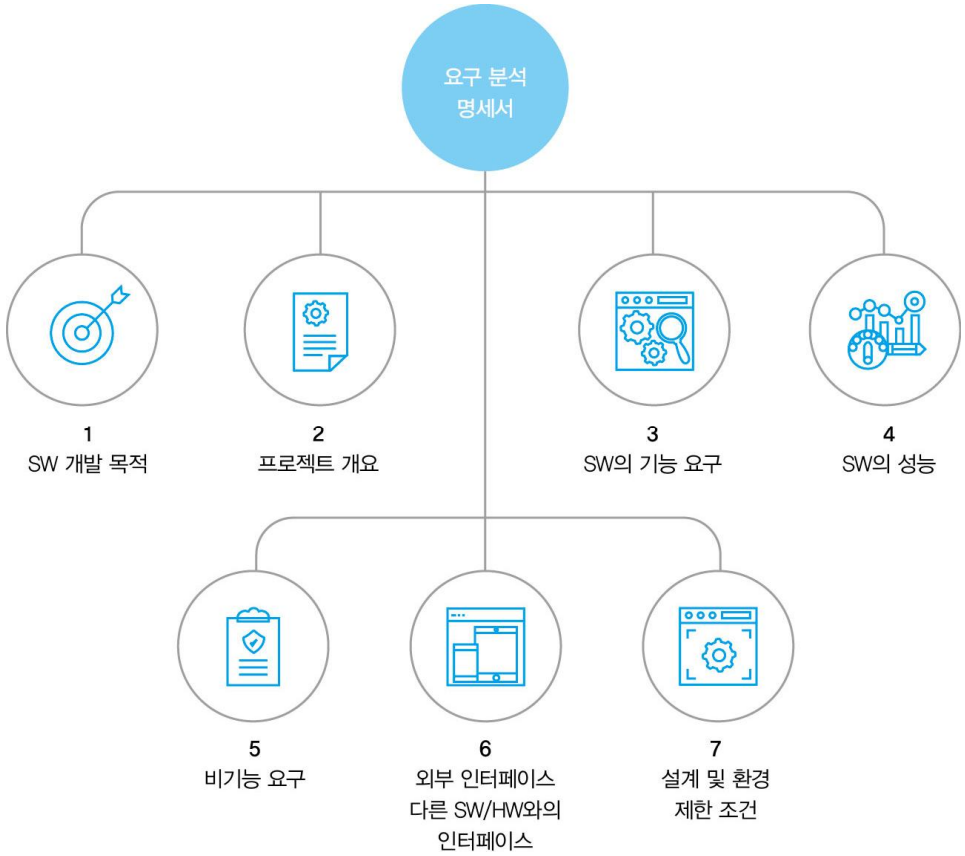
- 환자 진료 기록이 각 부서 간에 공유되는 흐름을 분석할 때
- 처방전 발행부터 약국 전달까지의 데이터 흐름을 설계할 때
- 의료 영상 데이터의 생성, 저장, 조회 프로세스를 분석할 때

유스케이스 다이어그램 사용이 적절한 상황

- 의사가 환자를 진료하고 처방하는 프로세스를 정의할 때
- 간호사가 환자 정보를 조회하고 기록하는 기능을 설계할 때
- 환자가 진료 예약을 하고 결과를 확인하는 시나리오를 정의할 때

요구 분석 명세서는 요구 분석 단계의 최종 산출물

- 목표, 범위, 기능/비기능 요구, 프로젝트 제약 조건, 도입 전환 계획 등이 포함되고, 자료 흐름도나 유스 케이스 명세서도 포함
- 고객과 개발자가 소프트웨어 스펙을 정한 것으로 모든 소프트웨어 개발의 일정, 예측, 설계, 구현, 테스트 작업의 기준이 되며, 이 문서를 근거로 개발 진행 상황을 파악하여 작업에 대한 의사 소통이 가능해짐



요구 분석 명세서의 내용

요구 분석 명세서에 포함되어야 할 자세한 내용은 우측 표에 나열

요구 분석 명세서에 반드시 넣어야 하는 것은 아니지만 작업 표준, 품질 관리를 기술하기도 함

- 작업 표준: 개발하면서 지켜나가야 할 규칙
- 품질관리: 프로그램을 테스트 하는 방법과 버그 발생이 줄어 릴리스 시점을 판단하는 지표 등을

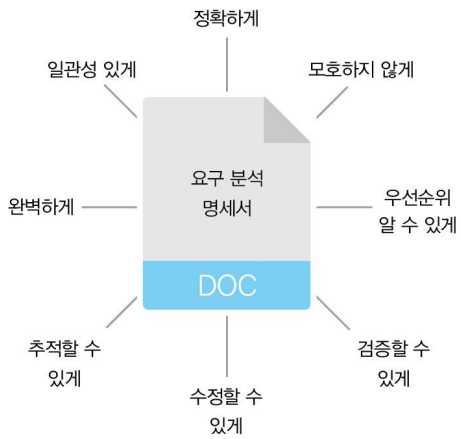
요구 분석 명세서의 내용

항목	내용
배경	SW로 개발할 대상 업무의 배경과 현황
과제	SW 개발 대상 업무 과제
목적 및 접근 방법	SW 개발하는 목적이나 과제의 해결 방법
기능	SW 시스템이 가지는 기능(기능 요구 사항과 비기능 요구)
시스템의 범위	SW 개발하려는 업무와 기능의 범위
사용자 인터페이스	SW 시스템을 사용할 때 사용자 인터페이스의 이미지
시스템 구성	SW 시스템의 하드웨어, 소프트웨어, 네트워크 구성 개요
도입, 이행 계획	SW의 도입 시기와 기존 시스템에서 마이그레이션하는 방법
운용, 유지 보수	시스템 운영 및 유지 보수 체제, 방법 등
용어 정의	시스템에서 사용하는 용어의 설명
계획	요구 확정, 설계, 개발, 테스트, 배포 등의 주요 작업의 완료 시기
환경	개발 진행 중의 인력 구조와 작업 환경 등
결과물	고객에게 제공하는 문서나 프로그램 등의 문서

요구 분석 명세서 작성시 다음 사항에 유의하면 읽기 쉽고, 개발 시 문제가 생기는 것을 줄여줄 수 있음

- 역할 분담
  - 책임 분담을 명확히 기술
  - 소프트웨어 기업이 담당해야 할 내용 이외에 고객이 스스로 해야 할 일이 있음, 가령 웹 콘텐츠 준비 등 고객이 분담할 일을 설명해야 추후 분쟁이나 오해를 방지
- 정량적
  - 배경과 개요를 정량적으로 작성하여 시스템을 둘러싼 상황을 명확히 알 수 있게 함
  - 기능과 성능도 정량적으로 작성하여 완성된 후 명세에 대하여 측정, 검증 확인할 수 있음
- 필요 충분
  - 해야 할 일 뿐 아니라, 하지 않아야 할 것도 명확히 기술
  - 명세서에 적혀 있지 않을 것을 하지 않은 것인지, 정하지 않은 것인지 명확히 해야 모호한 해석을 방지
- 현실감
  - 현재 상황(예산, 기간, 기술)에 맞게 실현될 수 있어야 함
  - 현존하지 않는 미래 기술에 의존해서는 안됨
- 구현 독립적
  - 작업을 완료할 수 있게 충분히 자세해야 하지만, 너무 자세하면 개발을 제한할 수 있으므로 너무 구체적이지 않아야 함
  - 특정 운영체제나 환경, 서비스에 의존하는 경우 설계가 제한됨

요구 분석 명세서 잘 작성하는 법



학습정리

- 기능요구 분석법은 소프트웨어 사용자의 요구를 이해하고 이를 데이터와 프로세스의 관점으로 변환하는 과정이다.
- 요구 분석은 데이터를 중심으로 이해하는 DFD가 가장 보편적으로 사용되며, 프로세스 명세서와 자료 사전을 통해 세부 사항을 보완한다.
- 유스 케이스 다이어그램은 시스템 내부와 외부로 관점을 나누고 요구사항을 수집하는 기법이며, 이를 통해 액터와 유즈 케이스를 식별한다.
- 자료 사전과 프로세스 명세서는 상세한 처리 내용과 조건을 구조적 언어로 기술한다.
- 요구 분석 명세서는 요구 분석의 최종 산출물로 개발과정에서 생길 수 있는 비효율을 최소화 하는데 기여한다.